

Análisis de Carga Computacional de ProtSpaM secuencial

Alumna: Ana Izaguirre
Asesor: Jorge González Domínguez

Octubre 2025

Índice

1. Contexto del proyecto	2
2. Adaptación de OpenMP a versión secuencial	2
3. Flujo principal (main.cpp): Pseudocódigo	4
4. Enlaces a código	5
5. Carga computacional por etapa	6
6. SpacedWords	7
7. Calc matches	7
8. Diagrama de flujo general versión secuencial	8
9. Referencias	8

1. Contexto del proyecto

ProtSpaM es una herramienta que compara secuencias biológicas, como proteínas o ADN, para medir qué tan diferentes o parecidas son entre sí. Estas diferencias se expresan como distancias evolutivas, que indican el grado de cambio o variación entre las secuencias, y ayudan a inferir qué tan relacionadas están biológicamente. En lugar de alinear completamente las secuencias (un proceso que puede ser lento), ProtSpaM utiliza un método llamado *spaced-words*, que identifica partes similares entre las secuencias de forma más rápida y eficiente, permitiendo analizar grandes volúmenes de datos.

El código base proviene del repositorio oficial **ProtSpaM (GitHub)**, desarrollado originalmente con paralelización en **OpenMP**. Como primera fase de este trabajo, se realizó un **fork** y se construyó una **versión secuencial**, en la que se eliminaron las directivas OpenMP, se verificó que los resultados fueran consistentes con la versión original y se aplicaron ajustes de limpieza y mejoras en la salida de información. Esta versión secuencial sirve como base para el análisis de carga computacional y para futuras paralelizaciones con **MPI**, cuyos enlaces en este documento corresponden a la rama con dicho código.

El flujo principal de ProtSpaM es el siguiente:

1. Carga de patrones (generados o desde archivo)
2. Parsing de secuencias (multifasta)
3. Cálculo de spaced-words por especie y patrón
4. Cálculo de matches entre todas las especies, generando la matriz de distancias
5. Salida de resultados en archivo

Objetivo: Analizar y documentar los puntos de mayor carga computacional dentro de ProtSpaM (versión secuencial).

2. Adaptación de OpenMP a versión secuencial

El código original de ProtSpaM estaba paralelizado con **OpenMP**. Para esta primera fase se construyó una versión secuencial con los siguientes cambios principales:

- Eliminación de las directivas de paralelización (**#pragma omp**) en bucles críticos.
- Verificación exhaustiva de que los resultados coincidieran con la versión OpenMP.

- Remover el parámetro thread (-t) del código, tanto de main como de los archivos que lo incluyeran.
- Limpieza de código y mejoras en la salida de información para facilitar el análisis.

Esta versión secuencial sirve como base para el análisis de carga computacional presentado en este informe.

3. Flujo principal (main.cpp): Pseudocódigo

Algorithm 1 Flujo principal de ProtSpaM

```

1: START
2: Inicializar temporizador global
3: Leer y parsear parámetros de entrada
4: ETAPA 1: Carga de patrones
5: if existe archivo de patrones then
6:   patterns = parsePatterns(archivo)
7: else
8:   patterns = rasbhari(patternNumber, weight, dc)
9: end if
10: if guardar patrones activado then
11:   guardar patterns en patterns.txt
12: end if
13: ETAPA 2: Parsing de secuencias
14: if entrada es archivo multifasta then
15:   species = parser(input_filename)
16: else
17:   species = sw_parser(inFiles, patterns)
18: end if
19: ETAPA 3: Cálculo de spaced-words
20: for cada especie  $s$  en species do
21:   for cada patrón  $p$  en patterns do
22:     spacedWords( $s$ ,  $p$ )
23:   end for
24: end for
25: ETAPA 4: Cálculo de distancias
26: Inicializar matriz distance[ $N$ ][ $N$ ]
27: for  $i = 0$  to  $N - 1$  do
28:   distance[ $i$ ][ $i$ ] = 0
29:   for  $j = N - 1$  down to  $i + 1$  do
30:     mismatch_rate = calc_matches(species[ $i$ ], species[ $j$ ], weight, dc, threshold, patterns)
31:     distance[ $i$ ][ $j$ ] = calc_distance(mismatch_rate)
32:     distance[ $j$ ][ $i$ ] = distance[ $i$ ][ $j$ ]
33:   end for
34: end for
35: ETAPA 5: Generación de matriz de salida
36: outputDistanceMatrix(species, output_filename, distance)
37: Mostrar tiempos de ejecución por etapa y total
38: END

```

4. Enlaces a código

- `main.cpp`: [GitHub](#)
- `calc_matches.cpp`: [GitHub](#)
- `misc.cpp` (`spacedWords`): [GitHub](#)

5. Carga computacional por etapa

Etapa	Función principal	Complejidad aproximada	Observaciones
Carga de patrones	<code>parsePatterns</code> <code>rasbhari</code>	$O(m \times w)$	Carga baja. Depende del número de patrones (m) y su peso (w).
Parsing de secuencias	<code>parser</code> <code>sw_parser</code>	$O(S \times n)$	Carga moderada. Escala linealmente con el número de especies (S) y la longitud promedio de las secuencias (n).
Cálculo de spaced-words	<code>spacedWords</code>	$O(S \times m \times n \times w)$	Carga intensiva. Etapa altamente paralelizable por especie o patrón. Representa uno de los puntos de mayor uso de CPU.
Cálculo de matches	<code>calc_matches</code>	$O(S^2 \times m \times n)$	Carga crítica. Es la fase más costosa debido a las comparaciones cuadráticas entre especies (S^2). Requiere estrategias de distribución de trabajo (MPI).
Generación de salida	<code>outputDistanceMatrix</code>	$O(S^2)$	Carga baja. Limitada principalmente por operaciones de E/S (I/O).

Cuadro 1: Estimación de la carga computacional por etapa en *ProtSpaM*. Donde S es el número de especies, m el número de patrones, w el peso del patrón y n la longitud media de las secuencias.

6. SpacedWords

ProtSpaM calcula *spaced-words* para cada especie y patrón. En este documento usamos la siguiente nomenclatura:

- S_i : la especie número i
- P_j : el patrón número j

Cada spaced-word se obtiene considerando únicamente las posiciones activas del patrón P_j sobre la secuencia de S_i .

Algorithm 2 Cálculo de spaced-words

```

1: for cada especie  $S_i$  en species do
2:   for cada patrón  $P_j$  en patterns do
3:     calcular spaced-words de  $S_i$  usando  $P_j$ 
4:   end for
5: end for

```

Paralelización sugerida: por especie (**MPI**). Cada proceso calcula spaced-words de un subconjunto de especies.

7. Calc matches

Una vez generados los spaced-words para cada especie S_i y patrón P_j , ProtSpaM compara todas las parejas de especies para calcular el *mismatch rate*, que refleja cuán diferentes están las secuencias.

En este documento usamos la siguiente nomenclatura:

- S_i : la especie número i
- S_j : la especie número j

Algorithm 3 Cálculo de matches

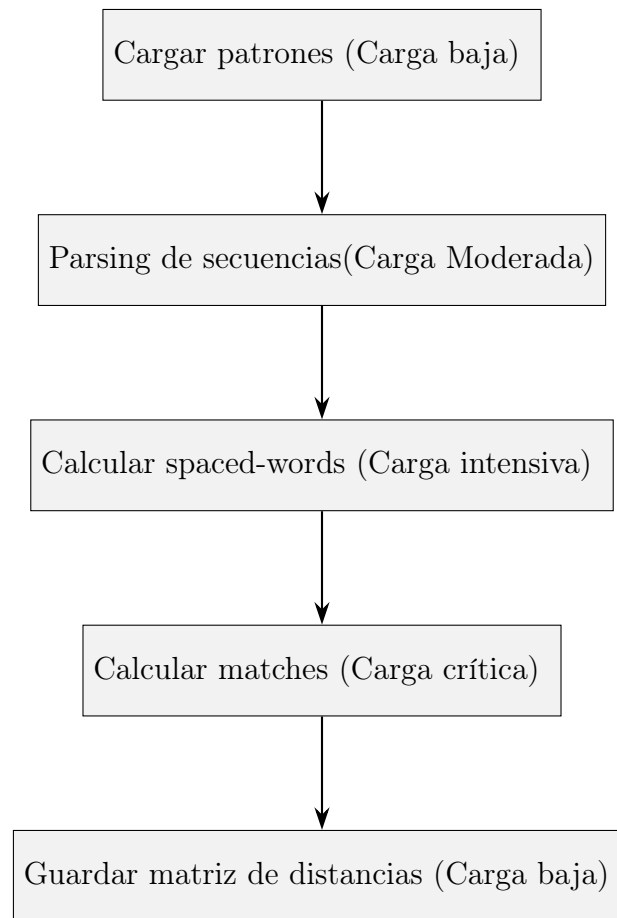
```

1: for cada especie  $S_i$  en species do
2:   Inicializar distancia  $distance[i][i] = 0$ 
3:   for cada especie  $S_j$  con  $j > i$  do
4:     Calcular el mismatch_rate entre  $S_i$  y  $S_j$  usando los spaced-words
5:     Convertir mismatch_rate en distancia y almacenar en  $distance[i][j]$ 
6:     Copiar la distancia simétrica:  $distance[j][i] = distance[i][j]$ 
7:     if mismatch_rate > umbral then
8:       Marcar como demasiado distante
9:     end if
10:   end for
11: end for

```

Paralelización sugerida: por pares de especies (**MPI**). Cada proceso calcula matches de un subconjunto de parejas.

8. Diagrama de flujo general versión secuencial



9. Referencias

- Schellhase, J. (2021). *ProtSpaM: Fast alignment-free phylogeny reconstruction from proteomes*. GitHub repository: <https://github.com/jschellh/ProtSpaM>
- Fork utilizado en este trabajo: <https://github.com/DevAna-2025/ProtSpaM>
rama utilizada: feat/seq.